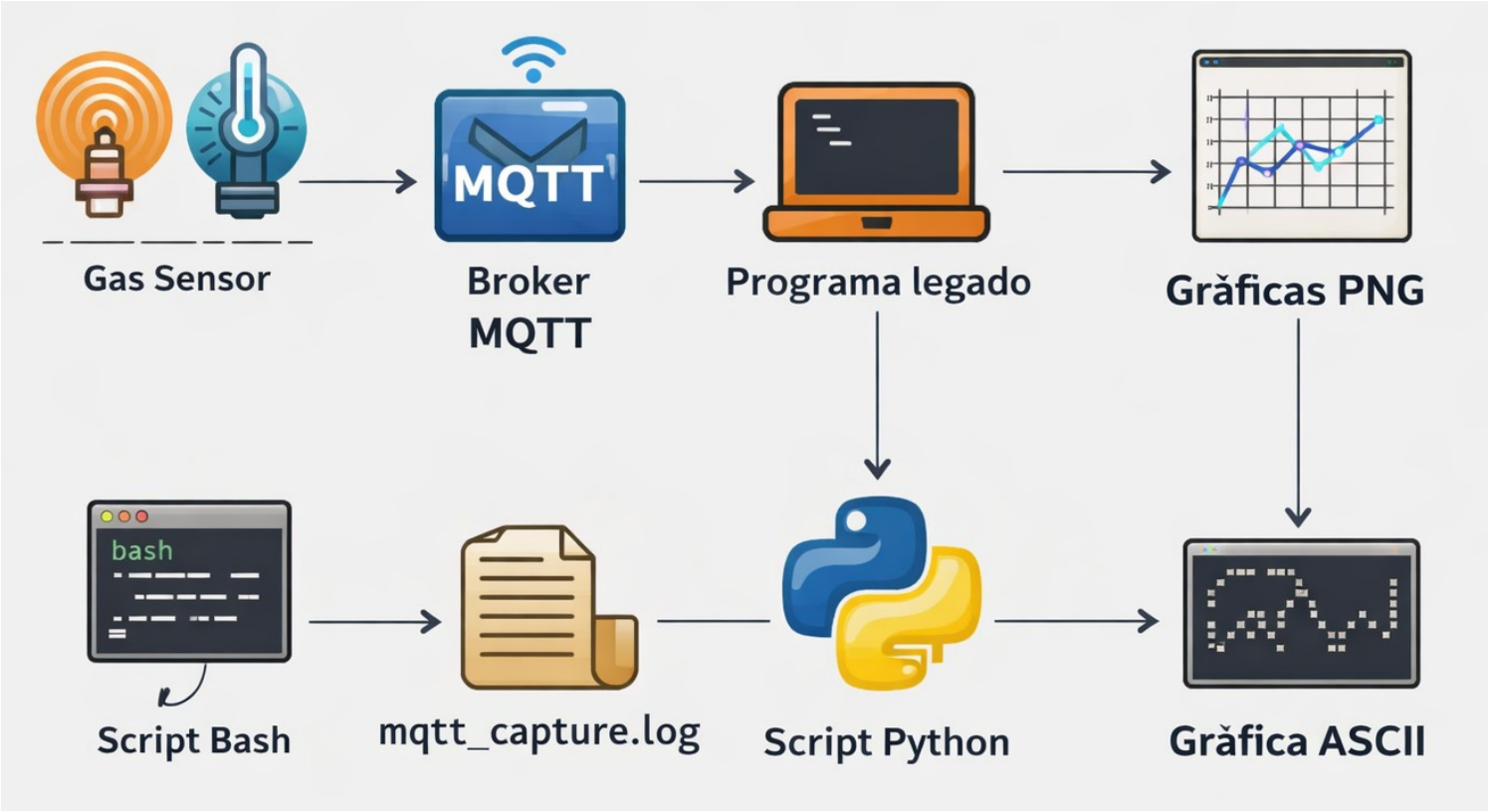
Repositorio



Práctica 1 -- Automatización con Bash y Python -- 25-26

Dimitry Company Cifre
Nidhal Majoul

índice

Introducción y contextualizacion del problema:.....3
Explicación del código:.....3
Explicación detallada del script Bash:.....3
Explicación detallada del script Python:.....5
Capturas de pantalla de ejecución:.....8
Explicación del control de procesos (PID y señales):.....10

Introducción y contextualizacion del problema:

En una empresa se tiene que trabajar con un programa compilado que realiza una conexión mediante MQTT a unos sensores, este programa por motivos de diseño obsoletos a día de hoy no almacena información ni en memoria ni en registros y medicucho menos tiene una API por lo que necesitamos realizar un script que recoja la salida del programa, las almacene y luego las gratifique para un mejor análisis. En este caso se pide en dos formatos Ascii para representarse en el shell y en una imagen .png
Para esto vamos a crear un Wrapper el cual se diseñara usando;
Bash para: lanzar, recolectar, guardar y detener el programa.
Python para: Formatear, maquetar y presentar los datos.

Explicación del código:

Explicación detallada del script Bash:

1. Aquí definimos dos tipos de salida, usando constantes para mejorar la legibilidad, definimos dos tiempo de espera para la finalización del programa, y definimos dos versiones de Paho que esperamos encontrar (1.3.2) la que se ha usado durante desarrollo y funciona y (1.3.) que seria cualquiera que debería funcionar.

```
#!/bin/bash
USER_DECLINED=3
DEPS_UNSATISFIED=4
MAX_SIGTERM=10
kill_timer=0
force=1
EXPECTED_PAHO_CPP_VERSION="1.3.2"
EXPECTED_PAHO_CPP_SERIES="1.3."
```

2. Iniciamos un bucle infinito preguntando por un numero, en caso de que no sea un numero valido (X<0) se presenta el mensaje de error correspondiente:

```
ask(){
  while true; do
    read -p "Introduzca el tiempo de captura (En segundos): " tiempo
    if [[ ! "$tiempo" =~ ^-?[0-9]+$ ]]; then
      echo "Eso no es un número entero válido."
      continue
    fi
    if (( tiempo < 0 )); then
      echo "El programa no puede esperar un numero negativo de tiempo"
    elif (( tiempo == 0 )); then
      echo "El programa no puede esperar cero segundos"
    else
      break
    fi
  done
}
```

3. Se pide al usuario el tiempo de llamada que se quiere recoger y se almacena en una variable llamada tiempo, posteriormente se avisa que se ejecutara el programa.

```
read -p "Introduzca el tiempo de captura (En segundos): " tiempo
```

4. Se ejecuta el programa en segundo plano mandando la salida a un archivo predefinido (mqtt_capture.log) tambien recogemos el PID y lo imprimimos usando $! que es una variable que almacena el PID del ultimo proceso en segundo plano ejecutado por eso necesitamos ejecutar el programa en segundo plano.

```
echo "[1/4] Ejecutando mqtt_subscribe_emqx_linux y guardando salida en mqtt_capture.log"
./mqtt_subscribe_emqx_linux > ./mqtt_capture.log &
PID=$!

echo "[info] PID=$PID"
```

5. Se espera el tiempo definido por el usuario y se manda la señal SIGTERM al programa para detenerlo, muestra un mensaje por consola.

```
sleep $tiempo
echo "[2/4] Finaliznado proceso (pid=$PID)"
kill $PID
```

6. Posterior mente se usa kill -0 "$PID" para saber si el proceso sigue vivo, insurreccionamos todas las salidas a null para que no molesten ya que solo queremos la señal de final de proceso del comando, en caso de que el proceso siga vivo se imprime un mensaje de aviso, mandamos la señal SIGINT para tener una segunda oportunidad de finalizar el proceso de forma controlada, reseteamos el contador de tiempo y cambiamos la variable force (para definir que la próxima vez sera necesario realizar una detención por fuerza bruta),

```
while kill -0 "$PID" >/dev/null 2>&1; do
if [[ $kill_timer -ge $MAX_SIGTERM && $force == 1 ]]; then
echo "[warn] Ejecutnado segundo metodo de finalizacion"
kill -2 "$PID"
force=0
kill_timer=0
```

7. En caso de que el contador vuelva a llegar a 0 y ya se aya mandado la segunda señal se usara SIGKILL como ultimo recurso para asegurarnos de que no quedara un proceso sin control en el sistema, ademas se imprimirá un mensaje de error avisando que el programa no se a finalizado como debería y que a partir de ahora el scrypt puede tener errores y la información del archivo .log puede encontrarse corrompida, fragmentada o no escrita.

```
elif [[ $kill_timer -ge $MAX_SIGTERM && $force == 0 ]]; then
echo "[error] Elimiando proceso se pueden producir erores..."
kill -9 "$PID"
fi
sleep 1
((kill_timer++))
done
Ejecutamos el código Python
python3 - << 'PY'
{Este bloque se ignora para reducir el tamaño de la explicación}
PY
}
```

8. Comprobamos que Pyhton este instalado en el sistema y en caso de que no este instalado se imprime un mensaje de error y se finaliza el script, por claridad la respuesta la mandamos a null porque solo con la correcta ejecución del programa ya es suficiente para confirmar la isntalacion

```
if command -v python3 >/dev/null 2>&1; then
else
echo "Este progrma necesita la instalacion de pyhton3"
exit 1
fi
```

9. Comprobamos si Python tiene las dependencias necesarias para su ejecución, si el comando no se ejecuta correctamente se continua en caso contrario se finaliza la ejecución.

```
if python3 -c "import matplotlib" >/dev/null 2>&1; then
echo "Dependencia python3 comprovada"
else
echo "Matplotlib no está instalada es necesaria para una correcta instalacion"
```

10. Comprobamos si libpaho-mqtt3cs esta instalado en caso de no encontrarlo o encontrar una versión diferente avisamos de que la versión no es la esperada pero ejecutamos el código por si la version instalada es compatible, por ultimo ejecutamos el main anteriormente explicado.

```
if ldconfig -p 2>/dev/null | grep -q libpaho-mqtt3cs; then
echo "Libreria libpaho-mqtt3cs detectada. Continuando..."
else
echo "[warn] No se encontro libpaho-mqtt3cs en el sistema"
fi
main
```

Nota: Este código no es compatible consecución se han eliminado tabulaciones y estructura por lo que este codigo no se puede ejecutar dentro de un entorno bash

Explicación detallada del script Python:

1. Primero tenemos los imports, estos son sys y json, estas son librería de pyhton3, que usaremos para salir del programa en caso de no poder ejecutar el código Python y crear el JSON. Junto con la definición de constantes que definen el nombre del archivo de entrada, el nombre del archivo de salida y la precisión del gráfico ascii.

```
import sys
import json
import matplotlib.pyplot as plt
LOG_FILE = "mqtt_capture.log"
OUTPUT_IMAGE = "Grafica.png"
PRECISION_GRAFICA=10
```

2. Posterior mente abrimos el archivo, en modo lectura, recoge todas las lineas y las devolvemos, en caso de no encortar el archivo se detienen la aplicación después de notificar al usuario.

```
def read_log(LOG_FILE):
try: with open(LOG_FILE, "r", encoding="utf-8") as file:
lines = file.readlines()
return lines
except FileNotFoundError:
print("El archivo log no existe.")
sys.exit(1)
```

3. Recogemos las lineas repasamos todas las lineas buscando el texto "Payload:" y guardamos todos los elementos que encontremos entre “{” y “}” como elementos como elementos JSON en caso de que una linea sea valida, pero no se pueda transformar en JSON simplemente la ignora maso y continuamos con la siguiere linea.

```
def extract_payloads(lines):
data = []
for line in lines:
if "Payload:" in line:
text_after = line.split("Payload:", 1)[1].strip()
start = text_after.find("{")
if start != -1:
end = text_after.find("}", start)
if end != -1:
possible_json = text_after[start:end+1]
try:
jsonLine = json.loads(possible_json)
data.append(jsonLine)
except json.JSONDecodeError:
pass #For now we don't do nothing with not valid json lines
return data
```

4. Generamos la imagen .png usando (matplotlib) comprobamos si los valores dentro del JSON son int’s o float’s usando la función isinstance() y se añade a la lista de all_values previamente inicializada, Usamos estos valores para generar el gráfico:

   figure(figsize=(12, 6)) inicia la configuración de la imagen con el tamaño.

   plt.plot introduce los valores a representar.

   plt.Title define el titulo.

   plt.Xlabel define el texto del eje X

   plt.Ylabel define el texto del eje Y

   plt.savefig Guarda la imagen .png

   plt.close Cierra el archivo.

```
def generar_grafico(data):
all_values = []
for item in data:
for val in item.values():
if isinstance(val, (int, float)): #We use the funtion isinstance() to check if the value (val) is an int or an float
to avoid use not numeric values
all_values.append(val)
plt.figure(figsize=(12, 6))
plt.plot(all_values)
plt.title("Todos los datos recogidos")
plt.xlabel("Índice")
plt.ylabel("Valor")
plt.grid(True)
plt.savefig(OUTPUT_IMAGE)
plt.close()
print(f"Gráfico guardado en: {OUTPUT_IMAGE}")
```

5. Recogemos los datos de la lista de elementos JSON y trabajamos con estos, en caso de que a la función no le lleguen datos se salta y sale.

```
def grafico_ascii(data):
vals = []
for d in data:
for v in d.values():
if isinstance(v, (int, float)):
vals.append(v)
if not vals: #Toda lista vacia en pyhton 3 se considra false negar un valor falso da positivo por lo que este
if solo saltara si no se ha guardado nada en vals
print("No hay datos para mostrar.")
return
```

6. Normalizamos el los valores que recibimos para representarlos con la precisión selecionada.

```
lines = []
for v in vals:
valor_escalado = int((v - min_val) / wide  * (PRECISION_GRAFICA - 1))
lines.append(valor_escalado)
```

7. Creamos un array bidimensional que usaremos como canva y iremos rellenandolo con los elementos que queremos dibujar.

```
width = n + 3
canvas = []
for _ in range(PRECISION_GRAFICA+1):
row = []
for _ in range(width+3):
row.append(" ")
canvas.append(row)
```

8. Dibujamos el carracter “*” en donde los datos indiquen que hay un punto.

```
for x, f in enumerate(lines):
y = PRECISION_GRAFICA - f
canvas[y][x+3] = "*"
```

9. Dibujamos una columna con el valor “|”

```
for y in range(PRECISION_GRAFICA+1):
canvas[y][3] = "|"
```

10. Dibujamos una fila con el valor “-”

```
for x in range(3, width):
    canvas[PRECISION_GRAFICA][x] = "-"
```

11. Dibujamos en la parte inferior izquierda un 0, en la parte superior del eje y el valor maximo que se ha alcanzado, y al final del eje x colocamos el numero de entradas que han llegado.

```
canvas[PRECISION_GRAFICA][width+1] = f"{len(vals)-3}"
  canvas[PRECISION_GRAFICA][0:3] = [" "," ", "0"]
  label_max = f"{max_val:.0f}"
```

12. Ejecutamos el código en un orden controlado pasando las variables atraves del mein

```
def main():
lines = read_log(LOG_FILE)
data = extract_payloads(lines)
generar_grafico(data)
grafico_ascii(data)
if __name__ == "__main__":
main()
```

Capturas de pantalla de ejecución:

Inicio:

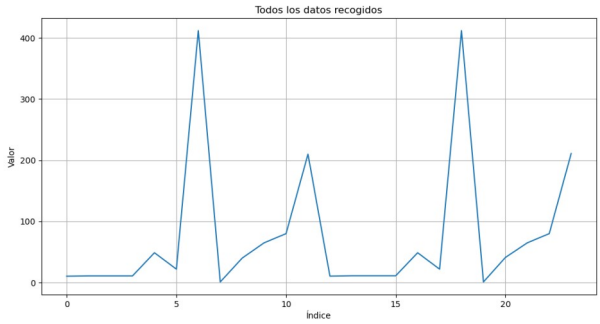
```
Dependencia python3 comprovada
Libreria libpaho-mqtt3cs detectada. Continuando...
Introduzca el tiempo de captura (En segundos): -1
El programa no puede esperar un numero negativo de tiempo
Introduzca el tiempo de captura (En segundos): 0
El programa no puede esperar cero segundos
Introduzca el tiempo de captura (En segundos): 10
[1/4] Ejecutando mqtt_subscribe_emqx_linux y guardando salida en mqtt_capture.log
[info] PID=614
```

Procedimiento:

```
[1/4] Ejecutando mqtt_subscribe_emqx_linux y guardando salida en mqtt_capture.log
[info] PID=614
[2/4] Finaliznado proceso (pid=614)
[3/4] Analizando datos
[4/4] Presentando graficos
Gráfico guardado en: Grafica.png
Imprimiendo grafico con 10 de precision
```

Resultado y Gráfico generado:

```
Imprimiendo grafico con 10 de precision
412|
   |     *          *
   |
   |
   |
   |
   |          *          *
   |
   |
   |   *    **     *    **
  0---------------------- 21
```



Explicación del control de procesos (PID y señales):

1. Se ha optado por implementar un mecanismo sencillo pero eficaz de control de procesos. El script concede al proceso dos oportunidades para finalizar de forma controlada. En caso de que esto no ocurra, se recurre finalmente al envío de una señal SIGKILL. Aunque esta señal fuerza la terminación inmediata del proceso, existe una alta probabilidad de que no provoque corrupción en los archivos ni en la información generada, por lo que los datos recopilados pueden seguir siendo válidos para su posterior procesamiento.
2. En primer lugar, el programa se ejecuta en segundo plano. Esto es necesario para que la variable especial del sistema $! se actualice correctamente y permita obtener el PID del proceso recién iniciado.
3. Transcurrido el tiempo especificado por el usuario, el script envía una señal SIGTERM con el objetivo de detener el proceso de manera controlada. A continuación, se concede un intervalo de gracia para permitir que el programa finalice sus operaciones pendientes.
4. Si tras dicho intervalo el proceso continúa activo, se asume la existencia de algún error o bloqueo. En ese caso, se envía una señal SIGKILL como último recurso. Asimismo, se notifica que el programa no ha finalizado correctamente, lo que podría ocasionar problemas como lecturas incorrectas de los datos o incluso la posible corrupción de los mismos.

Repositorio: UnInformaticoAburrido/Practicas_SO_2025_26 at Practica1